PROVEIT / DIOPHANTINE CONSISTENCY

New Structures for Shorter Lean Proofs

Four further proposals after eighteen earlier packets

Prepared for Vladimir Reshetnikov
26 October 2026

Main finding. The strongest remaining directions are in the newer Wang-tile geometry and in the formal identity behind the two-radicand combiner. Integer period closure replaces signed induction; injective exponent dilation replaces the choice of complex square roots; canonical band coordinates centralize helical uniqueness; shared edge fields make matching a consequence of the representation.

Baseline assumption. Every earlier suggestion in the six placed packets and twelve incoming archives is treated as already available, including optional and unselected ideas. None is counted as a new contribution merely because it has not yet been adopted.

Evidence. The packet includes Lean proof experiments, two reviewable production patches, compiler logs, source metrics, an exclusion audit and source/corpus hashes. The validation table distinguishes checked theorem cores from production integration. No complete project rebuild or repository-wide percentage saving is claimed.

Repository snapshot:

`553a5f7a463cbaab8800ff8148fef0bc5a9e1b22`

Lean: `4.32.0` mathlib: `81a5d257c8e410db227a6665ed08f64fea08e997`

Contents

1 Scope and conclusions 2

2 What was excluded and why 2

3 Use the helical period as a lattice vector 3

4 Prove the polynomial identity before evaluating it 5

5 Centralize the twisted band in one coordinate equivalence 6

6 Construct matching rows from shared edge fields 8

7 Validation and how to use the packet 10

8 References 11

1 Scope and conclusions

The user asks for additional ideas to shorten and simplify existing Lean proofs in `Computability/DiophantineConsistency`, while assuming the proposals in `research/lean-proof-simplification` and `docs/incoming` have already been applied. This changes the review criterion: an attractive refactor is not eligible if an earlier packet already describes its mechanism. Implementation work alone does not make an old suggestion new.

The examined snapshot has 445 Lean modules and 75,502 physical lines in the main `Lean/DiophantineConsistency` module directory. This count excludes its parent entry point and separate checking directories. It establishes the survey's scope; it is not a claim that all those declarations were reproved. The exact source hash is fixed throughout this report, so later changes to the live branch do not silently change the baseline.

Four directions survived the exclusion audit. They have different adoption costs and should not be presented as interchangeable kinds of progress.

Direction	What disappears	Appropriate interpretation
Integer period closure	Two signed-induction branches; a second seam-invariance induction	Small concrete replacement with exact source metrics
Formal polynomial descent	Chosen complex square roots and cast-back argument	Stronger polynomial theorem and simpler proof route
Canonical helical coordinates	Repeated quotient/remainder, divisibility and marker-coordinate arguments	Checked reusable interface; migration still required
Shared edge fields	Repeated horizontal and temporal matching obligations	Representation change with checked literal examples; total saving unmeasured

My recommended order is the first two local changes, followed by a focused experiment with the coordinate interface. The edge-field proposal has the largest potential to simplify subsequent construction proofs, but its adapter cost must be measured before a broad migration.

2 What was excluded and why

The audit covers all six placed packets `2026-10-26a` through `2026-10-26f`, consisting of 323 placed files, and all twelve incoming ZIP archives, consisting of 2,405 non-directory members. Complete paths, archive identities and SHA-256 hashes are recorded in `audit/corpus_manifest.json` and `audit/inventory.json`. The detailed mechanism-level audit is `audit/prior-art-audit.md`. Archive text was inspected without executing its programs.

The exclusion check includes report bodies, supplementary notes and optional candidate discussions. Bundled source snapshots were not mistaken for proposed changes. This distinction matters: several ideas missing from a report's headline table occur in its unselected-candidate notes.

The already-covered baseline includes list lookup APIs, scanner folds and iteration, primitive-recursive congruence, assignment transport, witness block composition, generic sum-of-squares extraction, heterogeneous block minima, generated circuit steps, degree certificates, sparse supports, coefficient bounds,

cyclic carries, Pell parity and growth, signed residue composition, exact rational-root descent, object-model interfaces, variadic proof fragments and normalized rule schedules. The following less obvious exclusions were also enforced:

- Support-restricted evaluator congruence and overwrite/frame rules.
- Certified normalization of the schema maps `rsep`, `rf1`, `rf2` and `ri`.
- Cyclic packing through additive-monoid algebra into $\mathbb{Z}/(b^n - 1)\mathbb{Z}$.
- Full-word canonicalization and fixed-length binary-digit injectivity.
- Explicit least and greatest cyclic carries, including two-solution ambiguity.
- Generic direct/native enumerator backends with preserved witness budgets.

A proposed exact binary-digit projection was discarded: once earlier full-word injectivity is assumed, its principal client is already easy, so its residual benefit is too small to justify another headline.

2.1 Novelty is relative to the inspected corpus

No matching proposal for the four directions below was found in the eighteen directly audited packets. This is a bounded repository-review claim, not a claim of mathematical priority or exhaustive historical search. In particular, periods of additive functions, injective polynomial substitution, mixed-radix coordinates and edge descriptions of Wang tilings are established ideas. The contribution is their precise application to these existing proofs after the earlier proposals are treated as complete.

New target	Closest earlier suggestion	Remaining distinction
Helical vector period	Finite backward-invariance arguments	Infinite integer lattice; direct integer multiples of a known period
Formal `core_eq`	Preserve a rational root during integral descent	No roots chosen; equality proved in the polynomial ring before evaluation
Band–time coordinates	Aligned `Cell` to `ZMod` equivalences	Two-dimensional twisted space–time quotient with period vectors $(w, -1), (0, H)$
Edge-field row construction	Guarded finite north/south color fibers	Construct whole compatible rows and cyclic grids; retain exact literal tiles

3 Use the helical period as a lattice vector

3.1 The current argument

In `WangTiles942HelicalQuotient.lean:22`, the theorem `row_zero_of_helix` assumes

$$T(x + w, t - 1) = T(x, t)$$

for all integers x, t, and proves

$$T(x, t) = T(x + wt, 0).$$

The current proof uses `Int.induction_on`, with separate positive and negative branches. Each branch shifts the spatial coordinate, invokes its induction hypothesis and normalizes an arithmetic expression. This is correct, but the invariant is already a period in an additive group.

3.2 The replacement

Define $F : \mathbb{Z}^2 \to \alpha$ by $F(x,t) = T(x,t)$. The hypothesis is exactly that $(w,-1)$ is a period of F. The library's closure of periods under integer multiples gives

$$F((x,t) + t(w,-1)) = F(x,t).$$

Since $(x,t) + t(w,-1) = (x + wt, 0)$, symmetry gives the result. This works for negative as well as positive t without another induction in the client.

The following exact replacement was checked in the pinned Lean environment:

```
theorem row_zero_of_helix {α : Type*}
    (T : ℤ → ℤ → α) (w : ℤ)
    (hhelix : ∀ x t, T (x+w) (t-1) = T x t)
    (x t : ℤ) : T x t = T (x+w*t) 0 := by
  have hp : Function.Periodic
      (fun p : ℤ × ℤ => T p.1 p.2) (w, -1) := by
    intro p
    simpa only [Prod.fst_add, Prod.snd_add, sub_eq_add_neg]
      using hhelix p.1 p.2
  simpa [zsmul_eq_mul, mul_comm]
    using (hp.zsmul t (x,t)).symm
```

The source patch uses the original statement layout. Its complete declaration shrinks from 19 to 7 physical lines, and its module from 147 to 135. The theorem statement is unchanged. No positivity, coprimality, vertical-period, allowed-tile or machine premise is introduced. The mathematical dependency is `Function.Periodic.zsmul`.

A second checked proof uses the shear $u \mapsto T(y - wu, u)$, which is one-periodic. It is included as an explanatory alternative. Only the shorter lattice proof belongs in the production patch; counting both as independent improvements would inflate the result.

3.3 Apply the same principle to seam status

`WangTiles942SeamColumns.lean:42` contains another signed induction. Its previous theorem already gives

$$\mathrm{SeamAt}(x, t+1) \iff \mathrm{SeamAt}(x,t).$$

Using propositional extensionality, this is period one for the Prop-valued function of time. `Periodic.int_mul_eq` gives equality with its value at zero, and hence the desired iff.

This declaration shrinks from 8 to 5 lines. Together, the two proposed module edits save **15 physical lines**, including the changed comments and without a new helper module. The value is modest but exact: two local integer inductions disappear.

The original and replacement helical experiments both report `[propext, Quot.sound]`. The standalone original seam lemma reports `[propext]`, while the periodic version reports `[propext, Quot.sound]`. A checked Boolean-valued variant has the same additional dependency. This is standard Lean library use, but the delta should remain visible if the project tracks minimal axiom dependencies. The full production seam dependency closure was not freshly rebuilt.

4 Prove the polynomial identity before evaluating it

4.1 Why complex square roots are avoidable here

The existing RefinedPair.lean:61 theorem `value_eq` identifies the ordinary two-radicand combining polynomial with an explicit formula. It chooses complex square roots of two integer radicands, applies the signed-product theorem, substitutes their squares and casts the result back to integers. Negative radicands are allowed; working over the complex numbers makes the existing argument correct.

The conclusion, however, is a universal polynomial identity. The same source development already supplies the information needed to prove that identity formally, before assigning numerical values. The existing `radicalProduct_eq` theorem is polymorphic over a commutative ring, so it can be used with a polynomial ring as its coefficient type.

Write

$$K = B(2D-1), \qquad T = Bn + C - K(C + V_0V_1)$$

and define

$$Q(A_0, A_1) = (T^2 + K^2A_0 - K^2V_0^2A_1)^2 - 4T^2K^2A_0.$$

Work in $R[A_0, A_1]$, where $R = \mathbb{Z}[V_0, V_1, n, B, C, D]$. Let δ fix coefficients and send A_i to A_i^2.

Proposition 1. *If P is the repository's two-radicand core polynomial, then $P = Q$.*

Proof. The existing theorem `expand_two_core` states that $\delta(P)$ is the full four-factor raw product. Apply `radicalProduct_eq` in the ring $R[A_0, A_1]$: the raw product is $Q(A_0^2, A_1^2) = \delta(Q)$. Thus $\delta(P) = \delta(Q)$. The map δ is injective, so $P = Q$. □

Injectivity is not a claim that squaring is injective on arbitrary ring elements. It is injectivity of the formal polynomial substitution that sends every exponent vector e to $2e$. Distinct monomials still have distinct exponent vectors, including over coefficient rings with zero divisors. Mathlib supplies this as `MvPolynomial.expand_injective`. Confusing these two notions of injectivity would invalidate the argument.

The repository facts used are in `RefinedRelationCombiningPolynomial.lean`. A small formal identity `core_eq` is followed by evaluation at the supplied integer parameters. The public `value_eq` proof then becomes an unfold, a rewrite by `core_eq`, and simplification.

4.2 What this improves

The proof now states its actual algebraic reason. It provides a stronger reusable polynomial identity and removes the local use of `IsAlgClosed.exists_pow_nat_eq`, the two chosen roots, their indexing proof, and the complex cast-back calculation. The explicit arithmetic formula and unrestricted integer inputs remain exactly the same.

This is different from the earlier rational-root descent proposal. There, a selected root is essential evidence and should be retained while proving it integral. Here, selecting roots is unnecessary because formal polynomial substitution already determines the identity.

In the prepared replacement, the direct Complex-polynomial import is removed from this module, but it is not claimed that the full transitive dependency closure no longer contains complex analysis. Nor does the change establish an axiom-free theorem: existing polynomial constructions may retain

`Classical.choice`. The auxiliary `core_eq` declaration must be charged when comparing source sizes. Compiler results and measured production-file metrics are in the evidence table and attached receipt.

The primary algebraic-only module retains the original finite-coordinate scaffolding. It changes 83 physical lines to 86, adding three lines in exchange for the stronger identity and simpler proof route. A separate experiment combined it with ordinary `finTwoArrowEquiv Bool` cleanup and reached 77 lines; that nine-line cleanup is not a new contribution and is not counted in the primary proposal. The algebraic patch is prepared but not compiler-validated: its dependency build stalled before the new module was reached. The mathematical derivation above and static data/interface checks are the evidence supplied.

5 Centralize the twisted band in one coordinate equivalence

5.1 The repeated arithmetic

The current construction extends a finite horizontal band to an integer plane, descends a helical plane to a cyclic word, and proves uniqueness of the marked tile. Its definitions occur in `WangTiles942HelicalExtension.lean:20`, `WangTiles942HelicalQuotient.lean:69`, and `WangTiles942HelicalMarker.lean:15`.

Fix positive w, H and an integer offset L. The canonical band is

$$I_{w,L} = \{x \in \mathbb{Z} : -L \le x < w - L\}.$$

The exact signed address used by the repository is

$$a(x,t) = -x - wt \pmod{wH}.$$

Its period vectors are $(w,-1)$ and $(0,H)$. These facts suggest one coordinate interface that can be reused by matching, tile binding and marker proofs.

Proposition 2 (Unique band and time class)**.** *For $x, y \in I_{w,L}$,*

$$a(x,t) = a(y,u) \quad \Longleftrightarrow \quad x = y \ \wedge \ t \equiv u \pmod{H}.$$

Proof. Equality of addresses gives

$$x - y + w(t-u) = wHk$$

for an integer k. Hence $x-y$ is a multiple of w. Since both spatial coordinates lie in the same half-open band of width w, $-w < x - y < w$. The only such multiple is zero, so $x = y$. Cancellation of the nonzero integer w then gives $t - u = Hk$. The converse is substitution. □

There is no coprimality hypothesis. This is ordinary integer cancellation and a unique representative in a half-open interval. It does not invert w inside $\mathbb{Z}/(wH)\mathbb{Z}$, where it is generally not a unit.

The checked core spells this out as:

```
theorem address_eq_iff (w H : ℕ) (hw : 0 < w)
    (L x y t u : ℤ)
    (hx : -L ≤ x ∧ x < (w : ℤ) - L)
    (hy : -L ≤ y ∧ y < (w : ℤ) - L) :
    address w H x t = address w H y u ↔
      x = y ∧ (t : ZMod H) = (u : ZMod H)
```

The actual source contains the complete proof. The three-theorem core in `lean/HelicalCoordinatesCore.lean` was checked successfully. The larger `lean/HelicalCoordinates.lean` additionally contains the prepared finite-coordinate equivalence; that extension received mathematical review but its compiler check did not finish under shared memory pressure.

5.2 Existence of coordinates and exact normalization

For any integer x, define

$$r = ((x + L) \bmod w) - L, \qquad q = \left\lfloor \frac{x + L}{w} \right\rfloor.$$

Then $x = r + wq$, so the normalization is

$$(x, t) \longmapsto (r, t + q), \qquad a(r, t + q) = a(x, t).$$

The plus sign in $t+q$ is essential. It matches the repository's extension and the period $(w, -1)$; reversing it would reverse the twist.

To obtain a representative of a finite address z, begin at $x = -z.\mathrm{val}$, $t = 0$, normalize by the formula above, and take the time class modulo H. The address returns exactly z. Combined with uniqueness, this gives

$$I_{w,L} \times \mathbb{Z}/H\mathbb{Z} \;\simeq\; \mathbb{Z}/(wH)\mathbb{Z}.$$

The prepared Lean extension gives injectivity and surjectivity proofs separately and defines `bandEquiv` using `Equiv.ofBijective`. That particular wrapper is noncomputable; the displayed quotient/remainder inverse is explicit and could instead be used as a computable inverse if desired. No computability improvement is claimed.

5.3 What the interface can remove

For $0 \le L < w$ and $x \in I_{w,L}$, the origin lies in the band and uniqueness specializes to

$$a(x, t) = 0 \quad \iff \quad x = 0 \;\wedge\; t \equiv 0 \pmod{H}.$$

This is the coordinate part of the marker argument. It can replace the custom chain in `zero_of_marker_coordinates`: a remainder equality, a reconstructed divisibility witness, a sign change in the quotient, and an interval divisibility-to-zero theorem.

The semantic marker premise must still be supplied:

$$C(x, t) = \mathrm{Start} \iff x = 0 \wedge t \equiv 0 \pmod{H}$$

for canonical-band sites. The geometry does not prove that premise for the machine construction. Once it is available, transporting it through the coordinate equivalence gives uniqueness of the marked word directly.

A spatial east step $x \mapsto x+1$ changes the cyclic address by -1; address successor $z \mapsto z+1$ moves west. Similarly, spatial horizontal motion becomes the ordinary successor inside the band and the successor with one time increment at its right boundary; vertical motion adds one to the time coordinate. These are the operations already established separately by `coordinates_succ_inside`, `coordinates_succ_wrap` and the quotient binding theorems. Consolidate those operations behind the equivalence only after checking that the actual signed address and tile values are preserved pointwise.

5.4 Scope and adoption cost

The address-equality theorem itself requires $w > 0$, but not $H > 0$. For $H = 0$, Lean's `ZMod 0` is the integers, and the theorem remains an infinite-coordinate statement. The finite equivalence explicitly requires both dimensions positive. The normalization identity also makes algebraic sense at $w = 0$, but provides no element of an empty band there. These edge cases are separated in the source and must remain separated in client prose.

The new library is larger than the single marker helper. Replacing only that helper is unlikely to justify the abstraction. The plausible payoff comes from using the same coordinates across extension, quotient binding, marker preservation and finite-word transport. Therefore this report claims a checked address core, a mathematically reviewed finite extension, and a concrete migration plan, not measured net production-line savings for this direction.

6 Construct matching rows from shared edge fields

6.1 Store each shared edge once

A current Wang tile has the field order west, north, east, south. For a row, choose three functions $E, N, S : \mathbb{Z} \to \mathbb{N}$, and define

$$R(x) = \langle E(x), N(x), E(x+1), S(x) \rangle.$$

Horizontal matching is now the identity

$$R(x+1).\text{west} = E(x+1) = R(x).\text{east}.$$

Its Lean proof is `rfl`. It no longer needs a separate argument for negative cells, the old head, the new head, and positions outside them.

This description is lossless: every horizontally matching row is recovered by taking its west, north and south fields. The supplied `row_eq_of_edges` and `row_fields_roundtrip` prove the two inverse laws between three arbitrary edge functions and a horizontally matching row. Their final lightweight implementation uses only Lean core imports.

For a time-index type σ with successor $s : \sigma \to \sigma$, choose $E, V : \sigma \to \mathbb{Z} \to \mathbb{N}$ and define

$$T(k, x) = \langle E(k, x), V(k, x), E(k, x+1), V(s(k), x) \rangle.$$

Now both horizontal and temporal matching are definitional:

$$T(k, x+1).\text{west} = T(k, x).\text{east}, \qquad T(s(k), x).\text{north} = T(k, x).\text{south}.$$

The time type may be finite cyclic. In particular, the reset-to-initializer edge can be represented by the same cyclic boundary function as every other temporal edge.

6.2 Where the current proofs repeat this work

`WangTiles942StepConstruction.lean:117` proves west and east signal formulas separately, then proves horizontal matching by analyzing the move direction. `WangTiles942FinishConstruction.lean:119` proves initializer matching by five spatial cases. `WangTiles942TraceCycle.lean:29` repeats the live/halted state split across four row properties and later proves the finite cycle's temporal matching by initialization, finish and last-row cases.

The matching parts of these arguments express equality of shared edges. Giving the shared edge a single definition changes the representation so that the equality no longer needs to be proved independently in every client. The genuine remaining work is that each resulting four-tuple is an allowed literal tile and has the required machine interpretation.

6.3 A concrete signal field for an operating step

Let the old head be h, let the move direction be d, and let ρ_d be the actual rail color for the transition's target. Define

$$E(x) = \begin{cases} \rho_d, & d = \text{right and } x = h + 1, \\ \rho_d, & d = \text{left and } x = h, \\ 0, & \text{otherwise.} \end{cases}$$

For a right move, the old head's east edge and the new head's west edge are both $E(h + 1)$. For a left move, the new head's east edge and the old head's west edge are both $E(h)$. This agrees with the production transition and receive tuples; no direction is reversed.

The kernel-checked `literalStep_eq_edges` experiment uses the exact transition/receive/pass pattern, with arbitrary rail colors, source color, written color, target-color function and old tape-color function. It proves all four tuple fields equal by local position and direction cases. It assumes neither old matching results nor a live-state premise. This is material: several existing step-tile interfaces are valid for arbitrary `cfg` and `q`, while semantic north/south identities require `cfg.state = some q`.

To preserve that distinction in production, use q-specific source and output colors in the literal bridge. Apply the live-state premise only when identifying those fields with `color cfg` and `color (machine.step cfg)`. Do not silently strengthen unrestricted public interfaces.

6.4 An exact initializer example

The current initializer has four regions and literal tuples:

Position	West	North	East	South
$x < 0$	2	434	2	0
$x = 0$	2	435	3	1
$x = 1$	3	436	4	4
$x > 1$	4	436	4	0

Take $E(x) = 2$ for $x \leq 0$, $E(1) = 3$, and $E(x) = 4$ for $x \geq 2$. Let N and S be the north and south columns of the table. The standalone prototype proves the resulting row exactly equal to the literal initializer formula, including the head's south color 4. Horizontal matching follows from the general constructor.

This experiment does not claim that changing the initializer alone saves lines. Its bridge can be longer than the original short matching theorem. It validates the exact representation and shows how one local classification can serve all four projections. A migration should amortize that bridge over the step, initialization, finish, reset and cyclic construction clients.

6.5 Avoid a circular refactor

The supplied `grid_eq_of_matching` theorem says that an already matching grid admits an edge-field representation. It is a completeness and transport fact, not evidence that the old matching proof can be deleted. Using it to define the new grid would simply hide the original proof in the construction.

For an actual simplification, define E, V independently, prove literal allowedness and exact equality to the intended row data, and then use `rfl` for matching. The independent initializer and step-pattern bridges are included precisely to demonstrate this noncircular route. Halt persistence, the actual decoded action, finite-table membership and $H \geq 3$ separation of initializer/finish/reset still need their proofs. The new representation does not certify any of those facts for free.

7 Validation and how to use the packet

The validation uses Lean 4.32.0. Three theorem groups have successful kernel checks; the larger production migrations are separated below. The compiler could not complete some broad dependency imports under shared memory pressure. A stalled build is not a proof failure, and it is also not a successful validation. No build-time comparison is inferred.

Artifact	Established evidence	Limit
`Periodicity.lean`	Original and replacement generic helix/seam theorems compile; axiom lists recorded	Full production modules and clients not rebuilt
`HelicalCoordinatesCore.lean`	Address uniqueness, normalization and band-origin criterion compile	Full finite-equivalence extension received mathematical review; compilation not completed
`EdgeFields.lean`	Complete generic core, exact initializer and unrestricted literal step-pattern bridges compile	Actual production Tile wrappers and 942-tile clients not rebuilt
`RefinedPairAlgebraic.lean`	Ordinary polynomial proof; source-authenticated dependencies; static formula/interface preservation	NOT compiler-validated; dependency build stopped before proposed module

The periodicity patch saves 15 physical production lines: 12 in `row_zero_of_helix` and 3 in `seam_iff_zero`. This is a measured proposed-source change, not a repository-wide reduction or a completed production integration. The algebraic-only source changes 83 to 86 lines. Its integer endpoint body changes 16 to 3 lines, but the new `core_eq` declaration occupies 12 lines, including its 7-line proof. The full module cost is therefore correctly charged as an addition of three lines.

For the edge core, `row_fields_roundtrip`, `grid_horizontal` and `grid_temporal` use no axioms. The function-extensional normal-form proofs report `Quot.sound`. The coordinate core reports `propext`, `Classical.choice` and `Quot.sound`. The periodicity axiom comparison is discussed in Section 3. No claim is made about the uncompiled algebraic module's final axiom list.

The complete literal edge-example file also compiled successfully. Its initializer and step-pattern equality proofs report `propext` and `Quot.sound`, with no `sorryAx`. The successful final output is in `evidence/edge-fields.log`. No complete 942-tile production dependency cone was rebuilt. Both proposed patches remain unapplied.

7.1 Replaying the checks

The archive's `README.md` gives exact commands, environment requirements and paths. Use Lean 4.32.0 and mathlib at the pinned commit. The coordinate core requires mathlib; the edge-field core has no external imports. The larger arithmetic edge examples import Lean core Omega. The algebraic module also uses the three authenticated Hilbert Tenth dependency files listed in the evidence manifest. The README lists the explicit replay commands for a prepared repository environment.

Axiom-print commands remain in the files. They are part of the evidence, not assumptions added by this proposal. No supplied proof relies on a new axiom declaration, `sorry`, `admit`, or `native_decide`. Standard axioms reported by the Lean kernel are recorded separately.

Exact production source savings are reported only where complete proposed files and a diff exist. An independently checked theorem with the same statement is stronger evidence than a pseudocode sketch, but it is still not a complete downstream rebuild. The files supplied here are proposals for review; they were not committed or pushed to the user's repository.

7.2 Recommended implementation sequence

1. Apply the vector-period replacement. Check the two production modules and their existing focused checking modules. Retain or decline the small seam extension according to the project's axiom-dependency preference.
2. Add the formal `core_eq` identity and replace `value_eq` by its evaluation. Retain the literal polynomial formula and its public name. Validate the affected carrier clients with the existing checks.
3. Introduce the helical coordinate interface in a small geometry module. First refactor the marker coordinate helper and quotient binding. Then measure total source size, including the new interface, before migrating all extension clients.
4. Prototype edge fields in one complete construction chain: initializer, operating step, halted pass, finish, reset, cycle. Preserve unrestricted literal statements and discharge semantic state premises only where they are already required. Count the equality bridges and allowedness cases.

The meaningful acceptance test is whether fewer independent mathematical arguments remain, with the same public statements and exact data bindings. Short tactic scripts alone are not sufficient evidence of that result.

8 References

1. ProveIt, pinned source tree: https://github.com/VladimirReshetnikov/ProveIt/tree/553a5f7a463cbaab8800ff8148fef0bc5a9e1b22/Computability/DiophantineConsistency.
2. Placed proposal collection and adoption record: https://github.com/VladimirReshetnikov/ProveIt/tree/553a5f7a463cbaab8800ff8148fef0bc5a9e1b22/Computability/DiophantineConsistency/research/lean-proof-simplification.
3. Incoming proposal corpus: https://github.com/VladimirReshetnikov/ProveIt/tree/553a5f7a463cbaab8800ff8148fef0bc5a9e1b22/docs/incoming.
4. Mathlib periodic functions and polynomial exponent expansion, at the pinned revision shown on the title page. Direct source links appear beside each theorem used.